Travel Tutorial

Stack & Architecture Reference

Tech Stack

Layer	Choice	Notes
Framework	Next.js 16 (App Router, Turbopack)	React 19, TypeScript, one process serves pages + API routes
Styling	Tailwind CSS v4 + shadcn-style components	cn utility, @base-ui/react primitives wrapped in thin styled components
Fonts	next/font — Geist (body), Satoshi (brand)	Satoshi self-hosted via next/font/local (not on Google Fonts)
Database	PostgreSQL via Supabase	Accessed only through Prisma — no raw SQL except a static health check
ORM	Prisma 6	DATABASE_URL = pooled/pgbouncer, DIRECT_URL = direct (migrations)
Auth	Auth.js (NextAuth v5 beta), Credentials, JWT	bcrypt hashing, timing-safe login (dummy-hash for unknown emails)
AI — primary	Groq (qwen/qwen3.8-27b)	Real function-calling for tool orchestration
AI — fallback	Google Gemini	Used only when Groq is configured but fails at request time
AI — no-key mode	Heuristic regex/keyword stub	Whole app runs and can be demoed with zero AI keys
Maps	Leaflet + OpenStreetMap tiles	No API key needed
Weather	Open-Meteo	Free, no API key
Places/hotels	Foursquare Places API	Free Core fields only; ratings surfaced via a Google Maps link instead
Testing	Vitest (unit), Playwright (E2E)	E2E spins up its own throwaway SQLite DB
CI	GitHub Actions	Lint → unit tests → build → E2E on every push/PR
Hosting	Vercel	trustHost: true is safe specifically because Vercel's proxy sets X-Forwarded-Host

High-Level Architecture

```
Browser
  |
  |- Server Components (pages) --- auth() session check --- Prisma --- Postgres
  |
  |- Client Components (chat, forms) --fetch--> API Routes (src/app/api/*)
  |                                                    |
```

```
  |                                                  |- Zod validation (every input)
  |                                                  |- Ownership check ({id, userId} on every query)
  |                                                  |- Rate limiting (in-memory, per-key sliding window)
  |                                                  `- business logic (src/lib/*)
  |
  `- Public routes (/, /login, /register, /shared/[token]) -- no auth required
```

Request-Protection Layers (Defense in Depth)

- proxy.ts — edge-level fast-path redirect for /dashboard, /chat, /trips, /profile
- Every Server Component page also does its own auth() + redirect("/login") — so even if the proxy matcher is wrong, no data leaks
- Every API route re-checks session.user.id and scopes every DB query to {id, userId} — never trusts a client-supplied user id
- AI tool calls (src/lib/ai/tools.ts) re-validate args with Zod and call assertOwnedTrip() regardless of what the model outputs — the LLM is never trusted as an authorization boundary

The AI / Tool-Calling Pipeline

This is the core mechanic of the app:

1. User sends a message → POST /api/chat (src/app/api/chat/route.ts)
2. extractTripUpdate() pulls structured fields (destination, dates, budget, interests) out of the message into TripContext — the single source of truth for what's known about the trip, persisted per ChatSession.context (JSON column)
3. decideAndRunTool() (src/lib/ai/agent.ts) — the orchestrator — decides whether the message needs a backend action: Groq function-calling picks a tool (real path), Gemini does the same if Groq fails (fallback), or keyword/regex heuristics run if no AI key is configured (stub). At most one tool call per turn.
4. The tool's result becomes a groundingNote string
5. streamAssistantReply() (src/lib/ai/chat-service.ts) builds the system prompt — trip context, personalization, and hard instructions (never invent stays, never state a rating, disclose known preferences inline, don't deflect general-knowledge questions) — and streams the reply back

Why Hotels Are Real but Most Itinerary Content Isn't

- **Hotels/stays**: live Foursquare API call, anchored to fixed lat/lon (text search was found to resolve “Goa” to Genova, Italy)
- **Weather**: live Open-Meteo call
- **Activities/attractions**: a curated static dataset (src/lib/planner/destinations.ts), ~15 destinations only
- **Itinerary structure**: a deterministic algorithm over that static data, not AI-generated

Data Model (Prisma)

```
User --+-- UserPreferences (1:1 -- travel style, food pref, interests)
       |
```

```
+-- Trip[] --+-- Itinerary[] -- Activity[]
|            +-- Expense[]
|            `-- shareToken (nullable -- public read-only link)
|
`-- ChatSession[] -- ChatMessage[]
         (ChatSession.tripId links a conversation to the trip it created)
```

Every child table cascades on delete from User, so wiping a user's data is a single User.delete().

Notable Features

- **Shareable trip links** — /shared/[token], a 192-bit random token is the entire auth check, no login required, view-only
- **Recent chats** — capped at 10 sessions per user, oldest hard-deleted on overflow
- **Personalization** — new chats seed from saved profile interests/food preference; the AI states them inline in answers instead of a separate UI panel

Security Posture (VAPT-Reviewed)

SQL injection, XSS, CSRF, clickjacking, IDOR, and prompt-injection-via-tool-args were all explicitly checked and hardened. Two documented, accepted limitations:

- Rate limiting is in-memory — correct on a single instance, weaker across Vercel's multiple serverless instances
- trustHost: true is only safe behind a trusted proxy (Vercel) — would need reconsidering on a different host

Environment Variables

See .env.example — DATABASE_URL / DIRECT_URL (Supabase), AUTH_SECRET, GROQ_API_KEY, GEMINI_API_KEY (optional), FOURSQUARE_API_KEY (optional).